



# CSS Grid Layouts - Complete Notes Basic to Advance

## 1. What is CSS Grid?

CSS Grid is a **2D layout system**, which means you can control both **rows and columns at the same time**. Flexbox is a 1D system (either row or column at one time), while Grid handles both dimensions together.

It is perfect for:

- Page layouts
- Dashboards
- Galleries
- Card grids

There are two main parts:

- **Grid Container** — the parent element with `display: grid`
- **Grid Items** — direct child elements placed inside the grid

---

## 2. Basic Setup

```
<div class="container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
```

```
<div>6</div>
</div>
```

```
.container {
  display:grid;
  grid-template-columns:100px100px100px;
  grid-template-rows:80px80px;
}
```

This creates:

- 3 columns of 100px
- 2 rows of 80px

Grid items automatically fill:

- Left to right
- Top to bottom

### 3. **fr** Unit — The Superpower of Grid

**fr** means **fractional unit**.

It divides the remaining available space.

```
.container {
  display:grid;
  grid-template-columns:1fr2fr1fr;
}
```

The layout divides into:

- 1 part
- 2 parts
- 1 part

Example:

If container width = 800px

Then:

- First column = 200px
- Second column = 400px
- Third column = 200px

You can also mix fixed and flexible sizes:

```
grid-template-columns: 200px 1fr 1fr;
```

- First column = fixed 200px
- Remaining space divided equally

## 4. `repeat()` — Avoid Repetition

Instead of writing:

```
1fr 1fr 1fr 1fr 1fr
```

Use:

```
grid-template-columns: repeat(5, 1fr);
```

Examples:

```
grid-template-columns: repeat(12, 1fr);
```

12 equal columns

```
grid-template-columns: repeat(3, 200px);
```

3 columns of 200px

Mixed example:

```
grid-template-columns: 100px repeat(3, 1fr) 100px;
```

## 5. **gap** — Space Between Grid Cells

Use **gap** instead of margins.

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 20px;  
}
```

Other variations:

```
row-gap: 10px;  
column-gap: 30px;  
gap: 10px 30px;
```

## 6. Grid Lines — Exact Item Placement

Grid has numbered lines.

Example:

A 3-column grid has 4 vertical lines.

You can place items using those lines.

```
.item {  
  grid-column-start: 1;  
  grid-column-end: 3;  
  
  grid-row-start: 1;  
  grid-row-end: 2;  
}
```

Shorthand:

```
.item {  
  grid-column: 1/3;  
  grid-row: 1/2;  
}
```

Using **span**

```
.item {  
  grid-column: 1/span2;  
}
```

Starts at line 1 and spans 2 columns.

## Negative Line Numbers

```
.item {  
  grid-column: 1/-1;  
}
```

This makes the item stretch full width.

- **1** means the last line.

## 7. **grid-template-areas** — Most Readable Layout Method

You can visually define layouts.

```
.container {  
  display: grid;  
  grid-template-columns: 200px 1fr 150px;  
  grid-template-rows: 80px 1fr 60px;  
  
  grid-template-areas:
```

```

"header header header"
"sidebar main      aside"
"footer footer footer";

    gap:10px;
}

```

Assign areas:

```

.header { grid-area:header; }
.sidebar { grid-area:sidebar; }
.main    { grid-area:main; }
.aside   { grid-area:aside; }
.footer  { grid-area:footer; }

```

Benefits:

- Very readable
- Easy responsive layouts
- No need to change HTML structure

Use `.` for empty cells.

## 8. `minmax()` — Flexible with Limits

```
grid-template-columns:minmax(200px, 1fr) 3fr;
```

Meaning:

- Minimum = 200px
- Maximum = 1fr

Example for rows:

```
grid-auto-rows:minmax(150px,auto);
```

Each row:

- Minimum 150px
- Grows if content increases

## 9. **auto-fit** and **auto-fill**

Best feature for responsive grids.

```
.gallery {  
  display:grid;  
  grid-template-columns:repeat(auto-fit,minmax(250px,1fr));  
  gap:20px;  
}
```

Meaning:

- Create as many columns as possible
- Minimum width = 250px
- Stretch equally

### Difference

#### **auto-fill**

Creates empty tracks if space exists.

#### **auto-fit**

Collapses empty tracks and stretches existing items.

Most of the time, use:

```
auto-fit
```

## 10. Implicit Grid

If items exceed defined rows/columns, Grid creates extra rows automatically.

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  
  grid-template-rows: 100px;  
  grid-auto-rows: 80px;  
}
```

Extra rows become 80px tall.

### grid-auto-flow

```
grid-auto-flow: row;
```

Default behavior:

Fill row by row.

```
grid-auto-flow: column;
```

Fill column by column.

---

## 11. Alignment in Grid

Two axes:

- `justify-*` → horizontal
- `align-*` → vertical

---

### Align Items Inside Cells

```
.container {  
  justify-items: center;  
  align-items: center;  
}
```



```
    place-items:center;
}
```

## Align Entire Grid Inside Container

```
.container {
    justify-content:center;
    align-content:center;

    place-content:center;
}
```

## Align Individual Item

```
.item {
    justify-self:end;
    align-self:start;

    place-self:startend;
}
```

## 12. Real Dashboard Example

```
<divclass="dashboard">
  <header>Logo & Nav</header>
  <nav>Sidebar</nav>
  <main>Main Content</main>
  <aside>Notifications</aside>
  <footer>Footer</footer>
</div>
```

```

.dashboard {
  display: grid;

  grid-template-columns: 220px 1fr 280px;
  grid-template-rows: 64px 1fr 48px;

  grid-template-areas:
    "header header header"
    "nav    main    aside"
    "footer footer footer";

  gap: 16px;
  min-height: 100vh;
  padding: 16px;
}

header { grid-area: header; }
nav     { grid-area: nav;   }
main    { grid-area: main;  }
aside   { grid-area: aside; }
footer  { grid-area: footer; }

```

## Responsive Version

```

@media (max-width: 768px) {
  .dashboard {
    grid-template-columns: 1fr;

    grid-template-areas:
      "header"
      "nav"
      "main"
      "aside"
      "footer";
  }
}

```

```
}  
}
```

## 13. Responsive Card Gallery

```
.cards {  
  display: grid;  
  
  grid-template-columns:  
  repeat(auto-fit, minmax(280px, 1fr));  
  
  gap: 24px;  
}  
  
.card {  
  background: white;  
  border-radius: 12px;  
  padding: 20px;  
}
```

Fully responsive without media queries.

## Quick Cheat Sheet

### Container Properties

```
display: grid;  
  
grid-template-columns  
grid-template-rows  
grid-template-areas  
  
gap
```

row-gap  
column-gap

grid-auto-rows  
grid-auto-columns  
grid-auto-flow

justify-items  
align-items  
place-items

justify-content  
align-content  
place-content

## Item Properties

grid-column  
grid-row

grid-area

justify-self  
align-self  
place-self

## Practice Tip

Use browser DevTools Grid Inspector.

- Firefox Grid Inspector is excellent
- Chrome also provides grid overlays

This helps visualize:

- Grid lines
- Gaps
- Areas
- Alignment

Very useful while learning Grid.